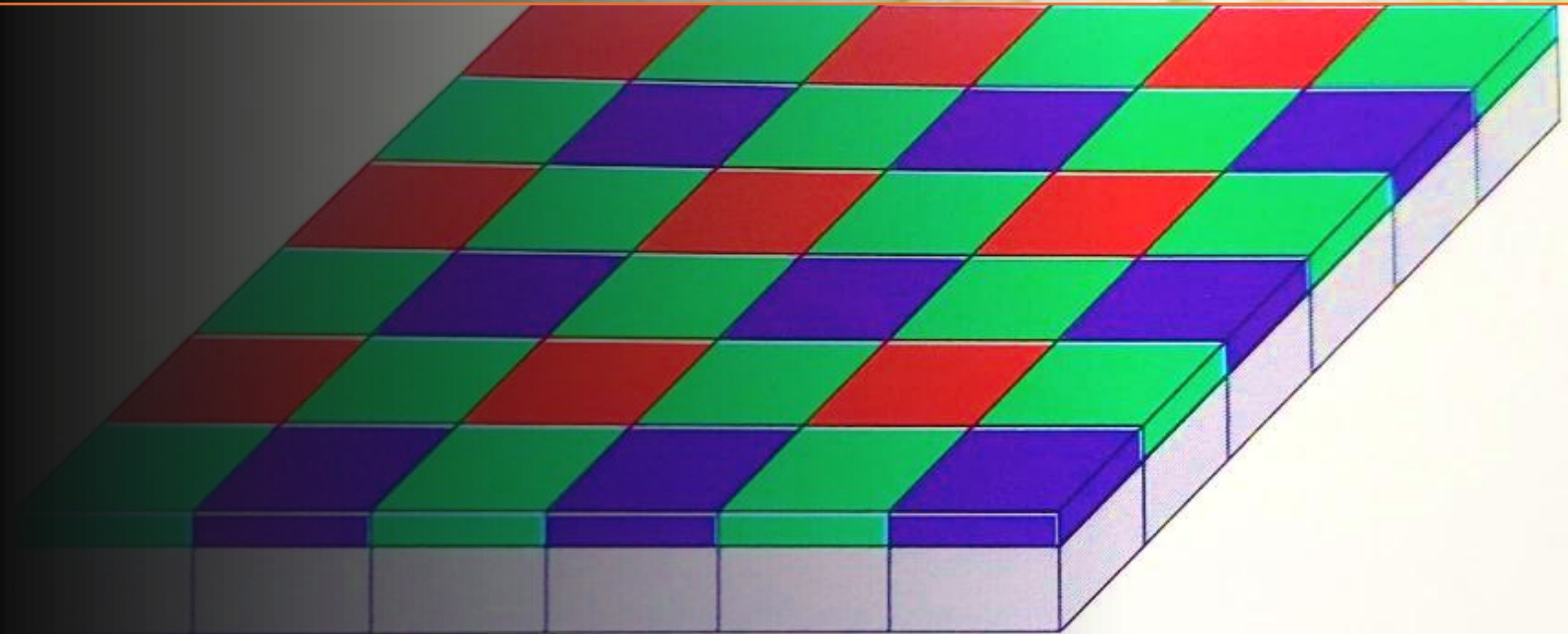


# M2. Visión Computacional

Clase 5

Arturo Cerón (Dr. Ing)

2026 @ITESM



# Agenda

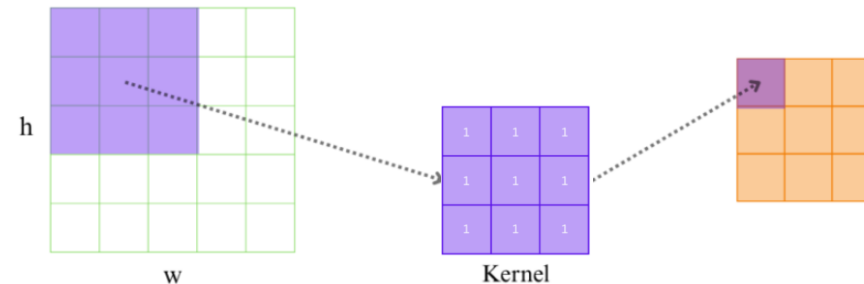
- 1. Repaso: Convolución
- 2. Matriz Hessiana
- 3. Detección de manchas
- 4. Compresión de imágenes
- 5. Detección de líneas y círculos
- 6. Algoritmo de Watershed
- 7. Flujo óptico

# 1. Repaso: Convolución

- Convolución: Es una operación matemática que combina a 2 funciones. En el caso del procesamiento de imágenes, una de las funciones es la imagen misma, y la otra es un filtro o “kernel”.
- Esto nos sirve para filtrar imágenes para diferentes propósitos, como:

- Suavizado
- Enfoque
- Detección de bordes
- Etc...

$$G(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k K(i, j) \cdot I(x - i, y - j)$$



**Fig3.** Convolution Operation

# 1. Repaso: Convolución

- ¿Qué representa una convolución?

$$314 \times 159 = 49926$$

$$\begin{array}{r} \times 314 \\ 159 \\ \hline 2826 \\ 15700 \\ 31400 \\ \hline 49926 \end{array}$$

$$\begin{array}{ccc} 159 & \longrightarrow & 951 \\ & & 314 \\ & \longrightarrow & 951 \end{array}$$

|   | 3  | 1 | 4  |
|---|----|---|----|
| 1 | 3  | 1 | 4  |
| 5 | 15 | 5 | 20 |
| 9 | 27 | 9 | 36 |

$$\begin{array}{r} 36 \\ 290 \\ 3600 \\ 16000 \\ 30000 \\ \hline 49926 \end{array}$$

# 1. Repaso: Convolución

- ¿Qué representa una convolución?

$P(\blacksquare + \blacksquare = 2) = a_1 \cdot b_1$   
 $P(\blacksquare + \blacksquare = 3) = a_1 \cdot b_2 + a_2 \cdot b_1$   
 $P(\blacksquare + \blacksquare = 4) = a_1 \cdot b_3 + a_2 \cdot b_2 + a_3 \cdot b_1$   
 $P(\blacksquare + \blacksquare = 5) = a_1 \cdot b_4 + a_2 \cdot b_3 + a_3 \cdot b_2 + a_4 \cdot b_1$   
 $P(\blacksquare + \blacksquare = 6) = a_1 \cdot b_5 + a_2 \cdot b_4 + a_3 \cdot b_3 + a_4 \cdot b_2 + a_5 \cdot b_1$   
 $P(\blacksquare + \blacksquare = 7) = a_1 \cdot b_6 + a_2 \cdot b_5 + a_3 \cdot b_4 + a_4 \cdot b_3 + a_5 \cdot b_2 + a_6 \cdot b_1$   
 $P(\blacksquare + \blacksquare = 8) = a_2 \cdot b_6 + a_3 \cdot b_5 + a_4 \cdot b_4 + a_5 \cdot b_3 + a_6 \cdot b_2$   
 $P(\blacksquare + \blacksquare = 9) = a_3 \cdot b_6 + a_4 \cdot b_5 + a_5 \cdot b_4 + a_6 \cdot b_3$   
 $P(\blacksquare + \blacksquare = 10) = a_4 \cdot b_6 + a_5 \cdot b_5 + a_6 \cdot b_4$   
 $P(\blacksquare + \blacksquare = 11) = a_5 \cdot b_6 + a_6 \cdot b_5$   
 $P(\blacksquare + \blacksquare = 12) = a_6 \cdot b_6$

Convolution of  $(a_i)$  and  $(b_i)$

|       | $a_1$           | $a_2$           | $a_3$           | $a_4$           | $a_5$           | $a_6$           |
|-------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|
| $b_1$ | $a_1 \cdot b_1$ | $a_2 \cdot b_1$ | $a_3 \cdot b_1$ | $a_4 \cdot b_1$ | $a_5 \cdot b_1$ | $a_6 \cdot b_1$ |
| $b_2$ | $a_1 \cdot b_2$ | $a_2 \cdot b_2$ | $a_3 \cdot b_2$ | $a_4 \cdot b_2$ | $a_5 \cdot b_2$ | $a_6 \cdot b_2$ |
| $b_3$ | $a_1 \cdot b_3$ | $a_2 \cdot b_3$ | $a_3 \cdot b_3$ | $a_4 \cdot b_3$ | $a_5 \cdot b_3$ | $a_6 \cdot b_3$ |
| $b_4$ | $a_1 \cdot b_4$ | $a_2 \cdot b_4$ | $a_3 \cdot b_4$ | $a_4 \cdot b_4$ | $a_5 \cdot b_4$ | $a_6 \cdot b_4$ |
| $b_5$ | $a_1 \cdot b_5$ | $a_2 \cdot b_5$ | $a_3 \cdot b_5$ | $a_4 \cdot b_5$ | $a_5 \cdot b_5$ | $a_6 \cdot b_5$ |
| $b_6$ | $a_1 \cdot b_6$ | $a_2 \cdot b_6$ | $a_3 \cdot b_6$ | $a_4 \cdot b_6$ | $a_5 \cdot b_6$ | $a_6 \cdot b_6$ |

- $A(x): a_5x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$
- $B(x): b_5x^5 + b_4x^4 + b_3x^3 + b_2x^2 + b_1x + b_0$

|          | $a_5x^5$       | $a_4x^4$    | $a_3x^3$    | $a_2x^2$    | $a_1x$      | $a_0$       |
|----------|----------------|-------------|-------------|-------------|-------------|-------------|
| $b_5x^5$ | $a_5b_5x^{10}$ | $a_4b_5x^9$ | $a_3b_5x^8$ | $a_2b_5x^7$ | $a_1b_5x^6$ | $a_0b_5x^5$ |
| $b_4x^4$ | $a_5b_4x^9$    | $a_4b_4x^8$ | $a_3b_4x^7$ | $a_2b_4x^6$ | $a_1b_4x^5$ | $a_0b_4x^4$ |
| $b_3x^3$ | $a_5b_3x^8$    | $a_4b_3x^7$ | $a_3b_3x^6$ | $a_2b_3x^5$ | $a_1b_3x^4$ | $a_0b_3x^3$ |
| $b_2x^2$ | $a_5b_2x^7$    | $a_4b_2x^6$ | $a_3b_2x^5$ | $a_2b_2x^4$ | $a_1b_2x^3$ | $a_0b_2x^2$ |
| $b_1x$   | $a_5b_1x^6$    | $a_4b_1x^5$ | $a_3b_1x^4$ | $a_2b_1x^3$ | $a_1b_1x^2$ | $a_0b_1x$   |
| $b_0$    | $a_5b_0x^5$    | $a_4b_0x^4$ | $a_3b_0x^3$ | $a_2b_0x^2$ | $a_1b_0x$   | $a_0b_0$    |

# 1. Repaso: Convolución

- ¿Qué representa una convolución?

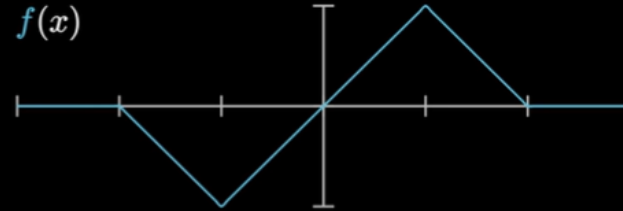
$$a = [1, 2, 3, 4]$$

$$b = [5, 6, 7, 8]$$

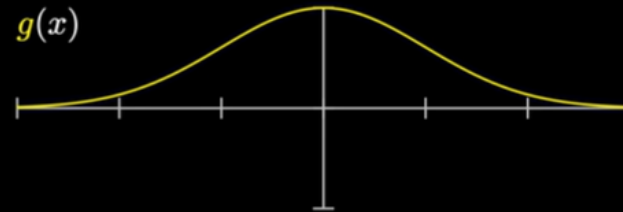
Convolution

$$a * b = [5, 16, 34, 60, 61, 52, 32]$$

$f(x)$

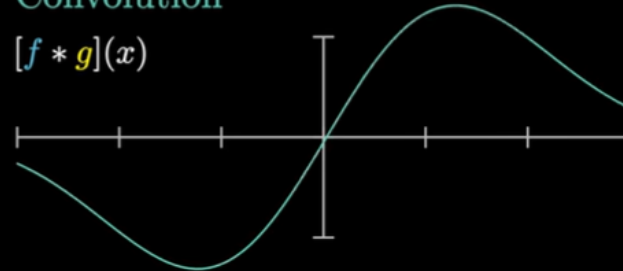


$g(x)$



Convolution

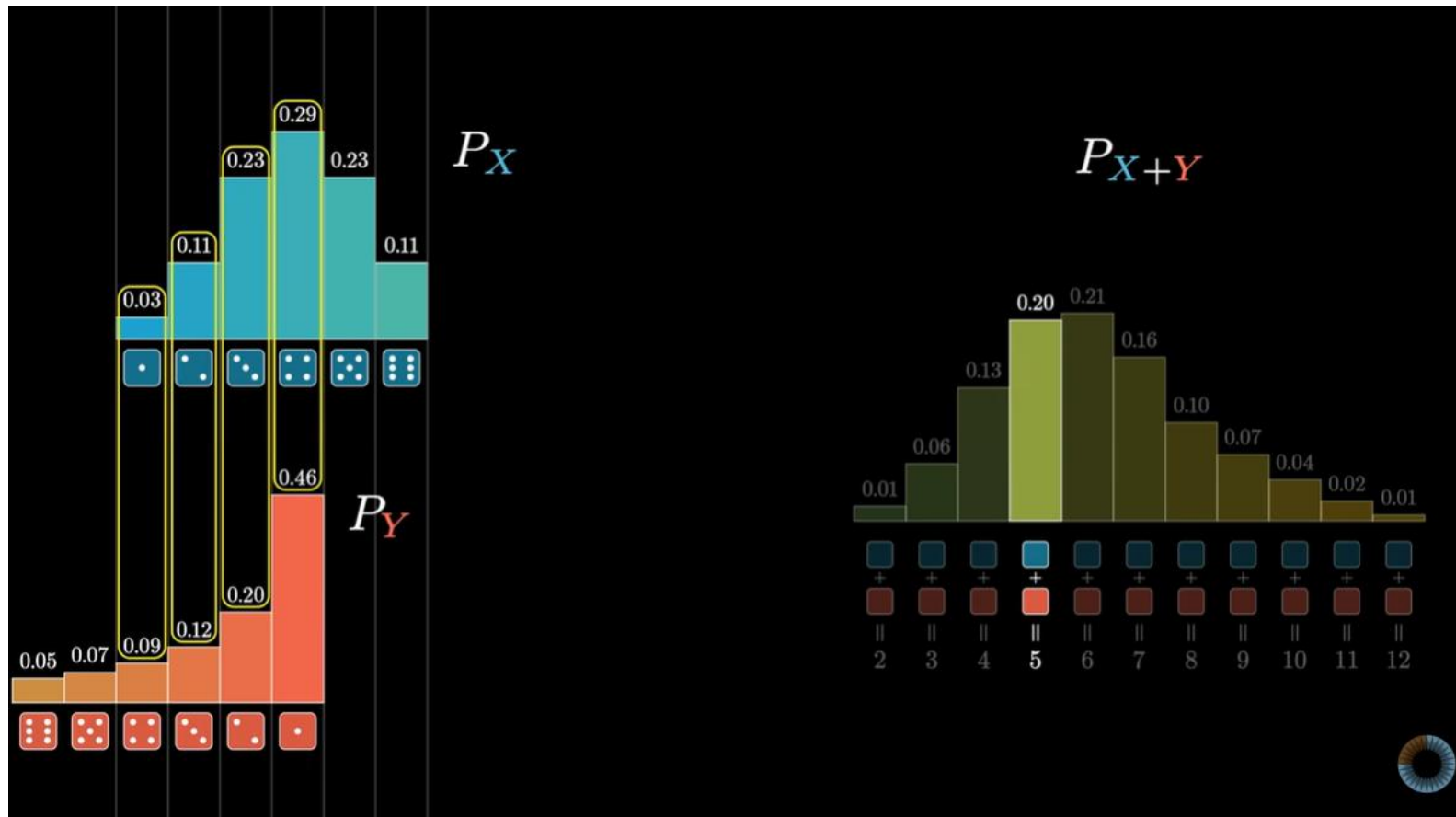
$[f * g](x)$





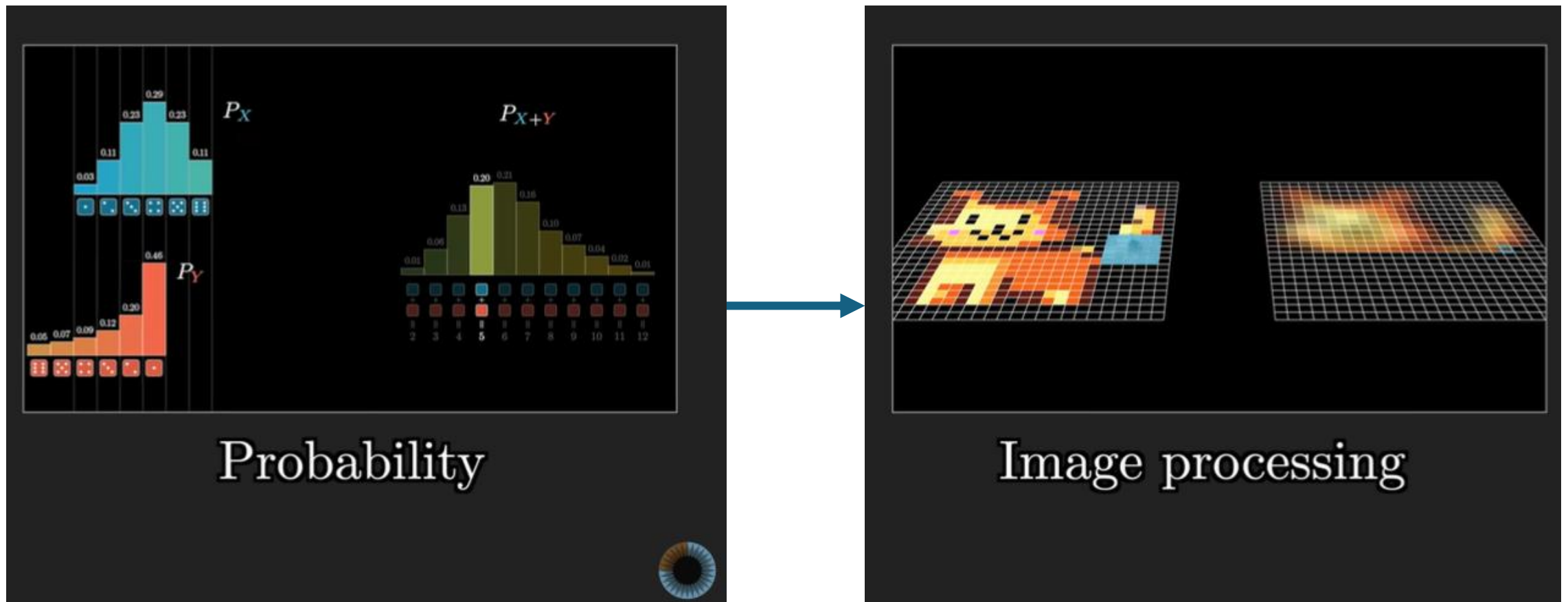
# 1. Repaso: Convolución

- Ejemplo: calculando probabilidades



# 1. Repaso: Convolución

- Ejemplo: filtrado de imágenes





# 1. Repaso: Convolución

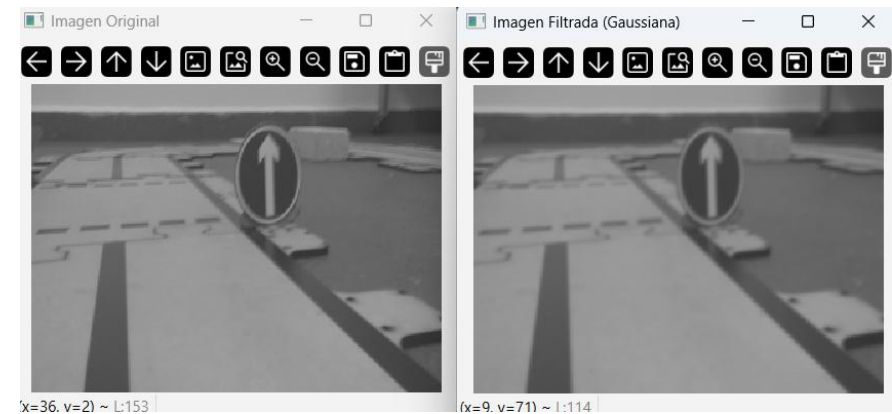
- Desenfoque (suavizado) y enfoque (afilado):

|                |   |   |   |
|----------------|---|---|---|
| $\frac{1}{16}$ | 1 | 2 | 1 |
|                | 2 | 4 | 2 |
|                | 1 | 2 | 1 |

3 x 3 Gaussian Kernel

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

**Fig6.** Sharpening Convolution Kernel



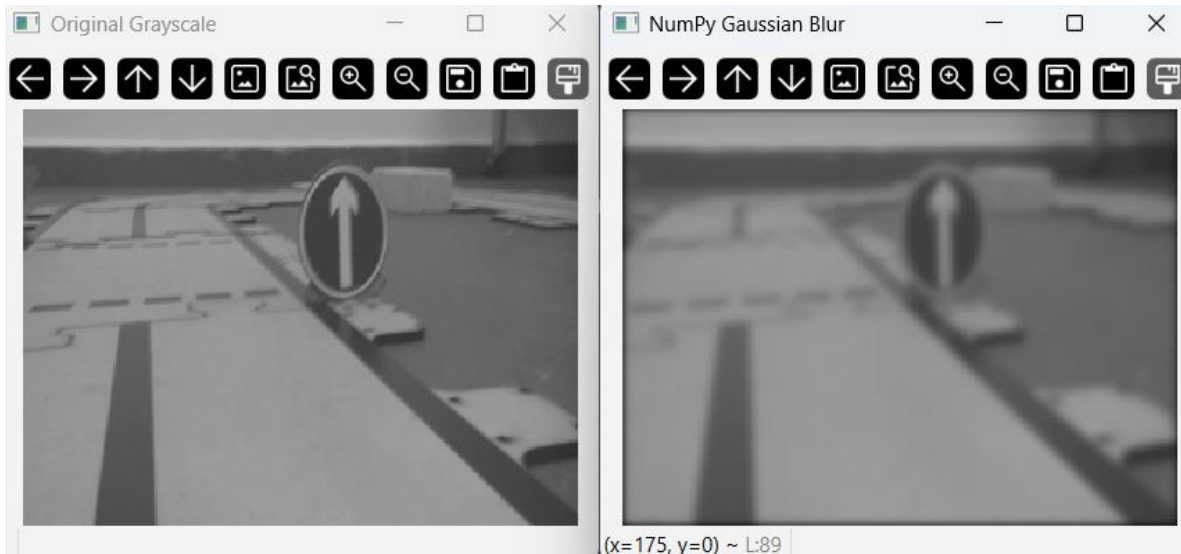
**Fig8.** Input Image



**Fig9.** Sharpened Image

# 1. Repaso: Convolución

- Convolución:



```
import cv2
import numpy as np

def manual_convolve(image, kernel):
    img_h, img_w = image.shape
    ker_h, ker_w = kernel.shape

    pad = ker_h // 2
    padded_img = np.pad(image, pad, mode='constant',
                        constant_values=0)

    output = np.zeros_like(image, dtype=np.float32)

    for y in range(img_h):
        for x in range(img_w):
            roi = padded_img[y : y + ker_h, x : x + ker_w]
            pixel_val = np.sum(roi * kernel)

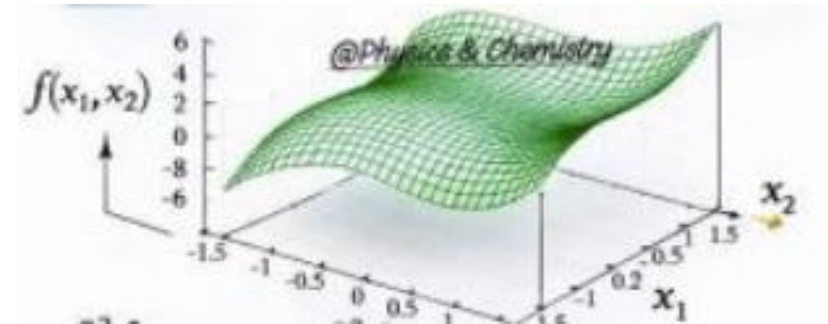
            output[y, x] = pixel_val

    return np.clip(output, 0, 255).astype(np.uint8)
```

## 2. Hessian Matrix

- Es una matriz cuadrada de derivadas parciales de segundo orden de un campo escalar (o función continua).
- En visión computacional, nos sirve para describir la curvatura local de la intensidad de la imagen.

$$H(f)(\mathbf{x}) = \nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$



$$H = \begin{bmatrix} I_{xx} & I_{xy} \\ I_{yx} & I_{yy} \end{bmatrix}$$

## 2. Hessian Matrix

- Se puede calcular con el **operador de Sobel** para las componentes horizontal y vertical. Es necesario aplicar el kernel horizontal 2 veces, y el vertical 2 veces.
- Para la componente mixta, se usa el kernel mostrado.

$$H = \begin{bmatrix} I_{xx} & I_{xy} \\ I_{yx} & I_{yy} \end{bmatrix}$$

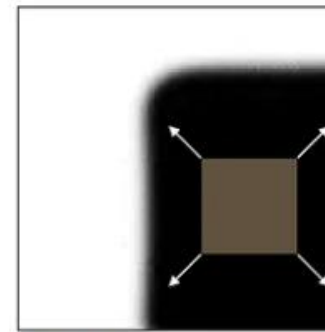
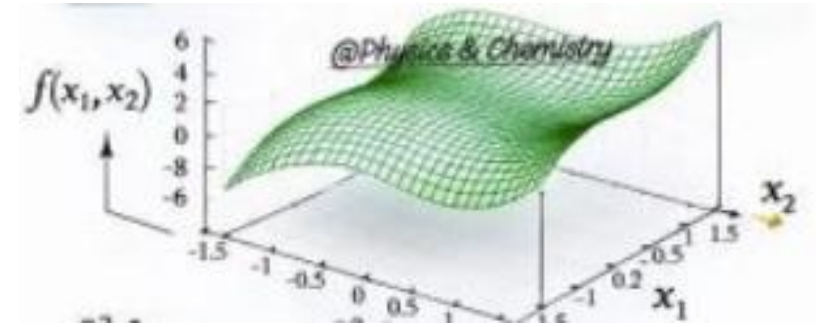
$$I_{xx} = \text{Image} \rightarrow G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \rightarrow G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

$$I_{yy} = \text{Image} \rightarrow G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} \rightarrow G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

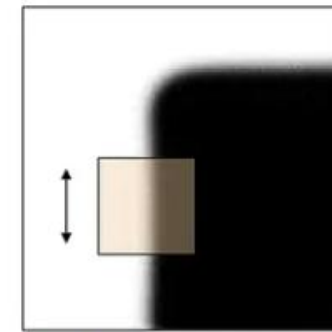
$$I_{xy} = I_{yx} = \text{Image} \rightarrow K_{xy} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

## 2. Hessian Matrix

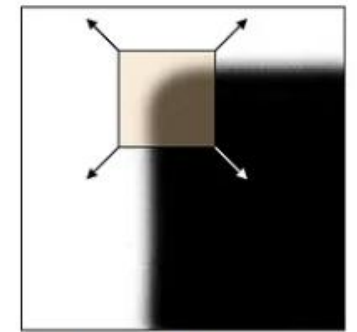
- Sus usos comunes incluyen detección de manchas (blobs) mediante la Determinante de la Hessiana.
- La discriminación entre esquinas y bordes mediante evaluación de curvaturas en distintas direcciones.
- Tiene relación con **el operador Laplaciano** (la Traza de la Matriz Hessiana).



"flat" region:  
no change in all  
directions



"edge":  
no change along  
the edge direction



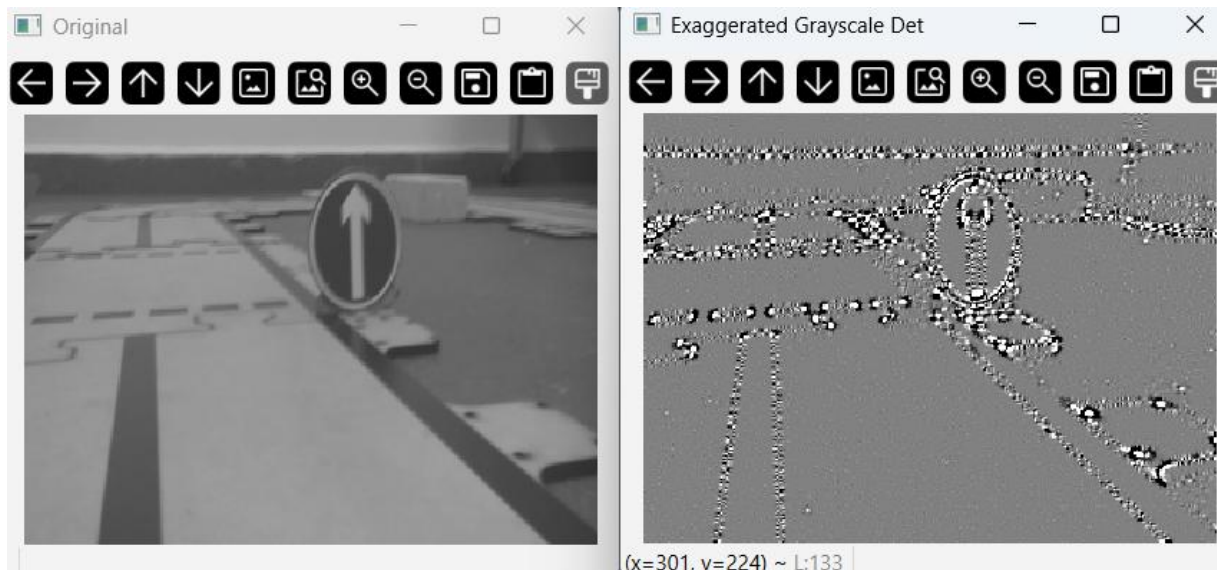
"corner":  
significant change  
in all directions

Figure 6. Illustration of three regions: flat, edge, and corner, from [9]

$$H = \begin{bmatrix} I_{xx} & I_{xy} \\ I_{yx} & I_{yy} \end{bmatrix} \quad \text{Tr}(H) = I_{xx} + I_{yy} = \nabla^2 I$$
$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

## 2. Hessian Matrix

- Determinante de la Matriz Hessiana:



```
import cv2
import numpy as np

def compute_hessian(image_path):
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    img_float = img.astype(np.float32)

    ix = cv2.Sobel(img_float, cv2.CV_32F, 2, 0, ksize=3)
    iy = cv2.Sobel(img_float, cv2.CV_32F, 0, 2, ksize=3)
    ixy = cv2.Sobel(img_float, cv2.CV_32F, 1, 1, ksize=3)

    det_hessian = (ix * iy) - (ixy**2)

    ix_vis = cv2.normalize(ix, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
    iy_vis = cv2.normalize(iy, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
    ixy_vis = cv2.normalize(ixy, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
    det_vis = cv2.normalize(det_hessian, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)

    cv2.imshow("Original", img)
    cv2.imshow("Determinant of Hessian", det_vis)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



### 3. Detección de manchas

- OpenCV cuenta con una función que considera varios parámetros para detectar manchas (blobs). Se llama “SimpleBlobDetector\_create”:
  - Umbral de intensidad
  - Área
  - Circularidad
  - Convexidad
  - Inercia
- Esta función suele ser más apta para flujos de trabajo en la industria.
- Existen algunas operaciones morfológicas que pueden resultar auxiliares en el proceso: **Dilatación y Erosión**

<https://www.geeksforgeeks.org/python/erosion-dilation-images-using-opencv-python/>

<https://learnopencv.com/blob-detection-using-opencv-python-c/>

Original Image



Original

After Dilation



After Dilation

After Erosion

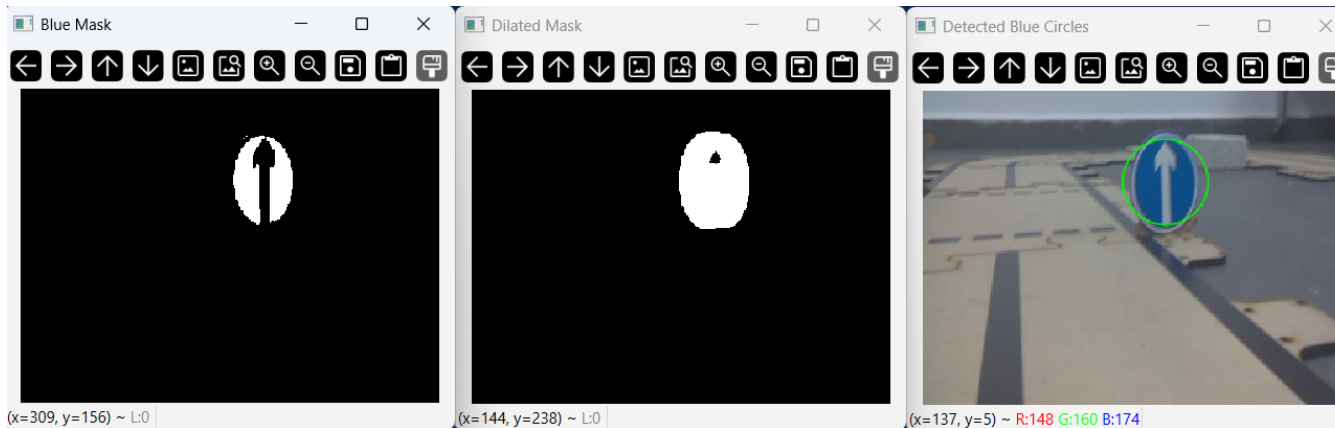


After Erosion

Se barren las imágenes en una ventana de píxeles y se toman los píxeles máximos (dilatación) o mínimos (erosión) de los vecinos.

# 3. Detección de manchas

- SimpleBlobDetector:
  - Aplicando dilatación



```
import cv2
import numpy as np

frame = cv2.imread('image.png')

hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

lower_blue = np.array([100, 150, 50])
upper_blue = np.array([140, 255, 255])

mask = cv2.inRange(hsv, lower_blue, upper_blue)
blurred_mask = cv2.GaussianBlur(mask, (7, 7), 0)
kernel = np.ones((9, 9), np.uint8)
dilated_mask = cv2.dilate(blurred_mask, kernel, iterations=1)
_, final_mask = cv2.threshold(dilated_mask, 127, 255,
cv2.THRESH_BINARY)

params = cv2.SimpleBlobDetector_Params()

params.minThreshold = 10
params.maxThreshold = 250
params.thresholdStep = 10
params.filterByColor = True
params.blobColor = 255
params.filterByArea = True
params.minArea = 50
params.maxArea = 1000000
params.filterByCircularity = False
params.filterByConvexity = False
params.filterByInertia = False
params.minCircularity = 0.5

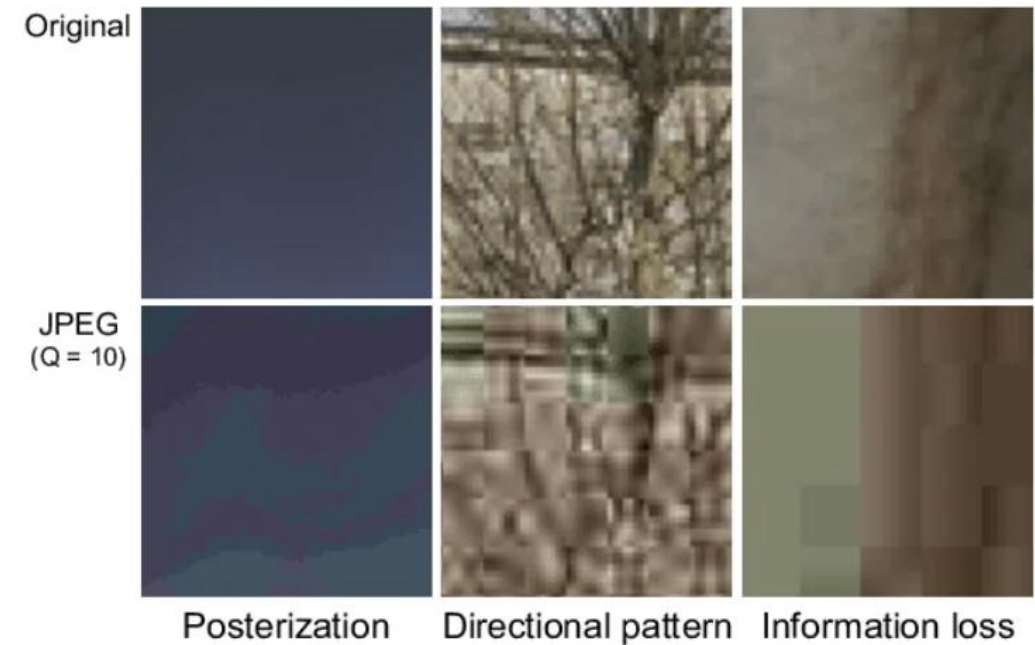
detector = cv2.SimpleBlobDetector_create(params)
keypoints = detector.detect(final_mask)

result = cv2.drawKeypoints(frame, keypoints, np.array([]), (0, 255, 0),
cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

cv2.imshow("Blue Mask", mask)
cv2.imshow("Dilated Mask", final_mask)
cv2.imshow("Detected Blue Circles", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

# 4. Compresión de imágenes

- En ocasiones, sólo nos interesa analizar la información “interesante” de una imagen, por lo que podemos descartar las partes poco interesantes. Por lo tanto, no es siempre requerido enviar la información completa de la imagen. Podemos enviar sólo las partes interesantes de la imagen, de manera que se pueda expresar las mismas ideas, pero con menos datos.
- Es posible que al descomprimir la imagen, se pierda algo de fidelidad con respecto a los datos originales. Por lo que dependiendo del uso, se deberá utilizar la técnica de compresión adecuada.
- ¿Pero cómo identificar la información interesante?



# 4. Compresión de imágenes

- Fast Fourier Transform (FFT)

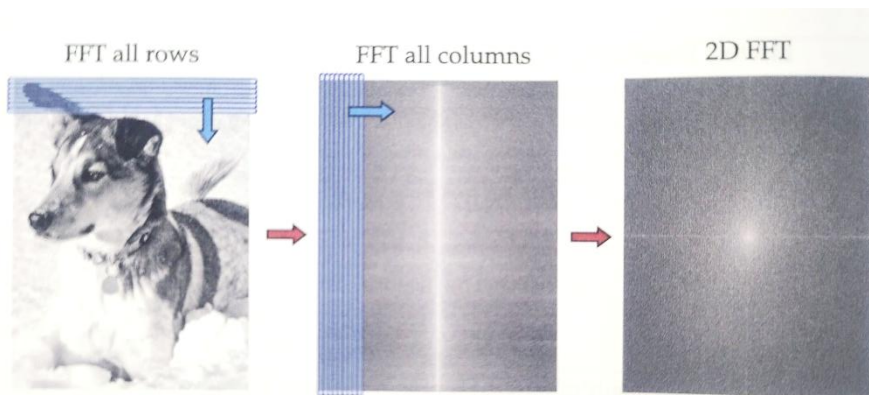
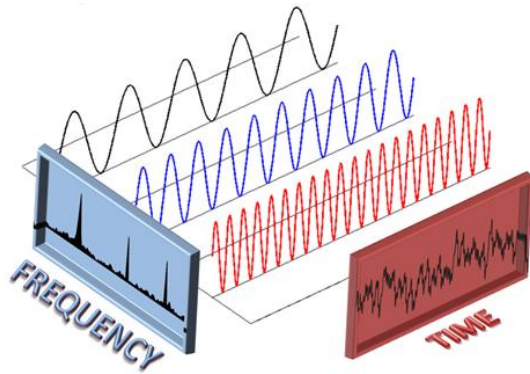


Figure 2.25 Schematic of 2D FFT. First, the FFT is taken of each row, and then the FFT is taken of each column of the resulting transformed matrix.



Figure 2.26 Compressed image using various thresholds to keep 5%, 1%, and 0.2% of the largest Fourier coefficients.

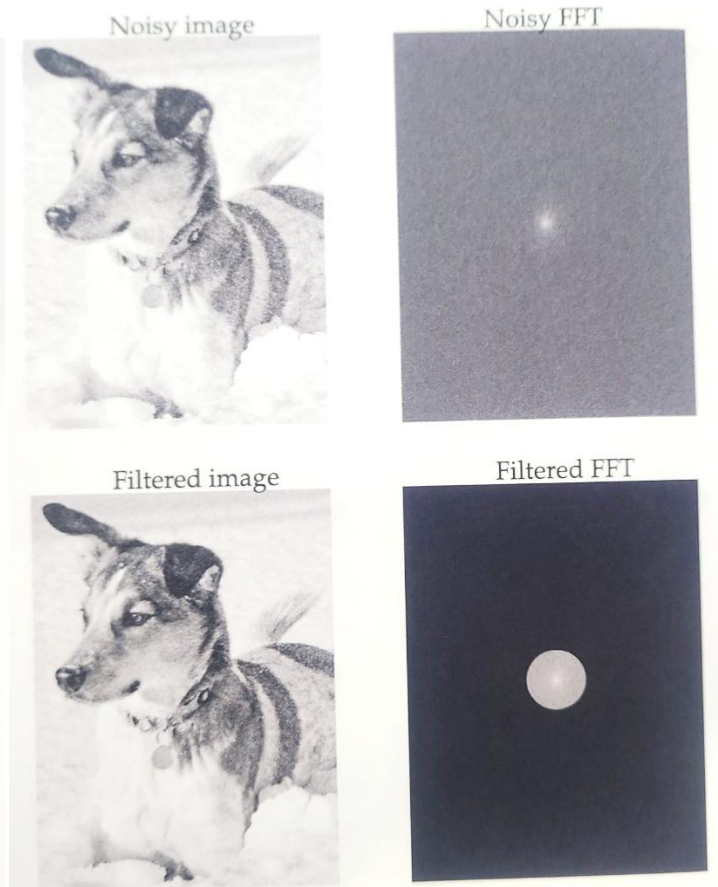


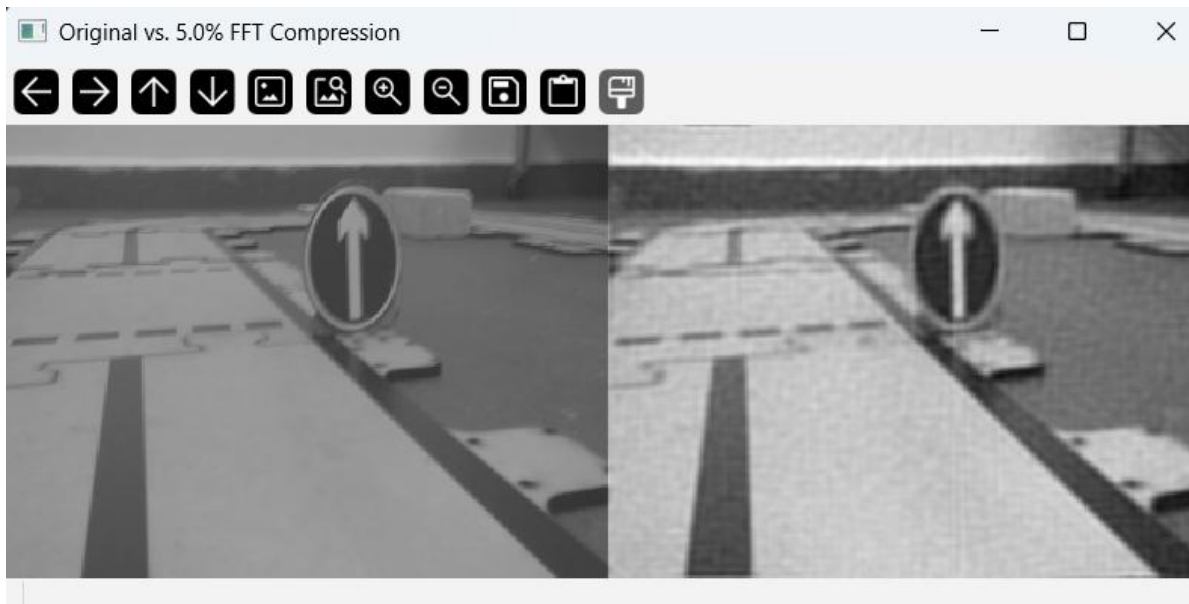
Figure 2.27 Denoising image by eliminating high-frequency Fourier coefficients outside of a given radius (bottom right).

[https://www.youtube.com/watch?v=toj\\_loCQE-4](https://www.youtube.com/watch?v=toj_loCQE-4)



## 4. Compresión de imágenes

- FFT al 5%
  - De 75KB a 30KB:



```
import cv2
import numpy as np

img = cv2.imread('image.png', cv2.IMREAD_GRAYSCALE)

img_float32 = np.float32(img)

dft = cv2.dft(img_float32,
flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft)

magnitude = cv2.magnitude(dft_shift[:, :, 0],
dft_shift[:, :, 1])
flat_mag = magnitude.flatten()
flat_mag.sort()

keep_fraction = 0.05

thresh_idx = int(len(flat_mag) * (1 - keep_fraction))
threshold = flat_mag[thresh_idx]

mask = magnitude > threshold
dft_shift[mask == 0] = 0

# Inverse FFT to get back to pixels
dft_ishift = np.fft.ifftshift(dft_shift)
img_back = cv2.idft(dft_ishift)

img_back = cv2.magnitude(img_back[:, :, 0],
img_back[:, :, 1])

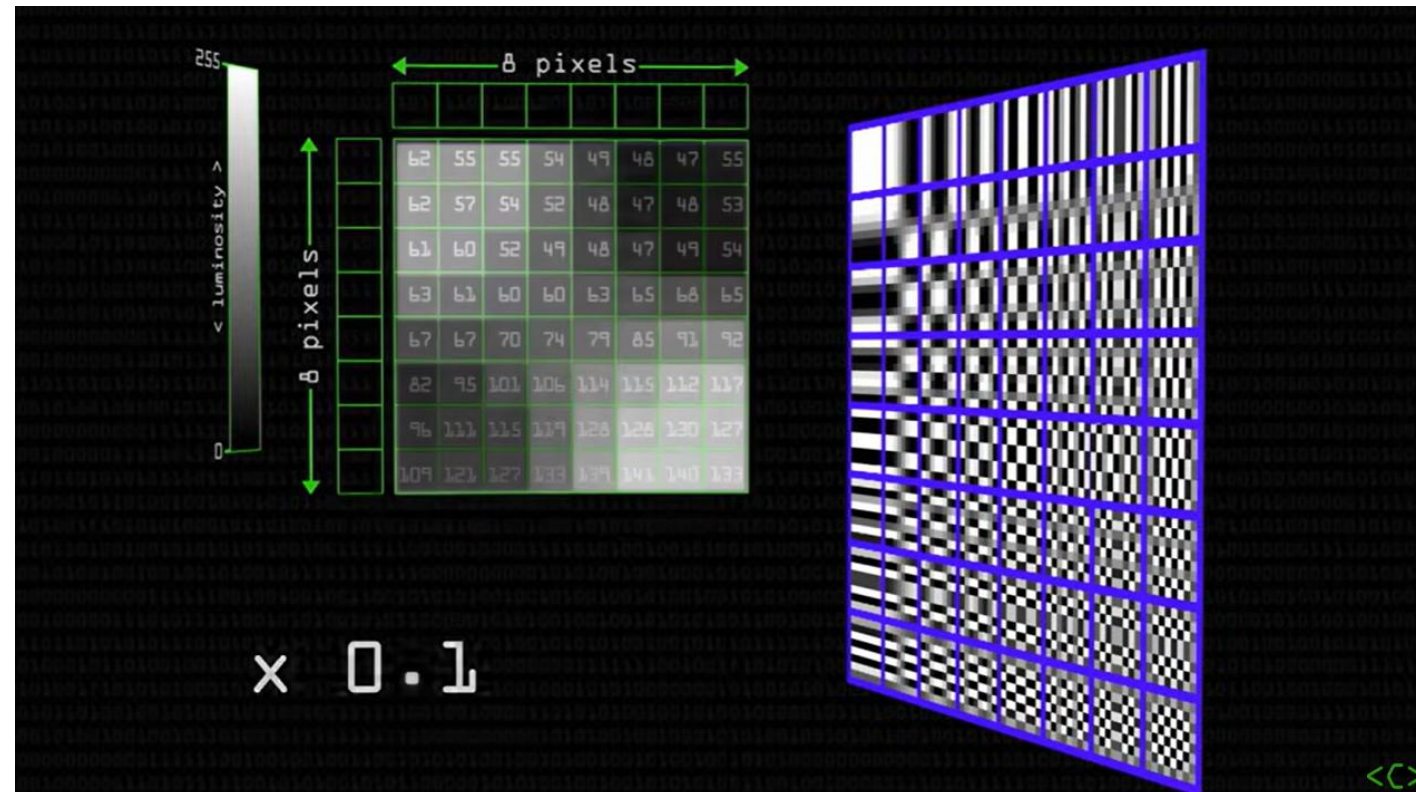
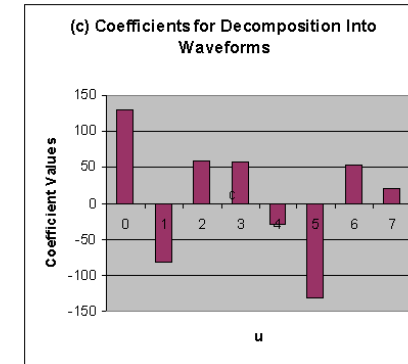
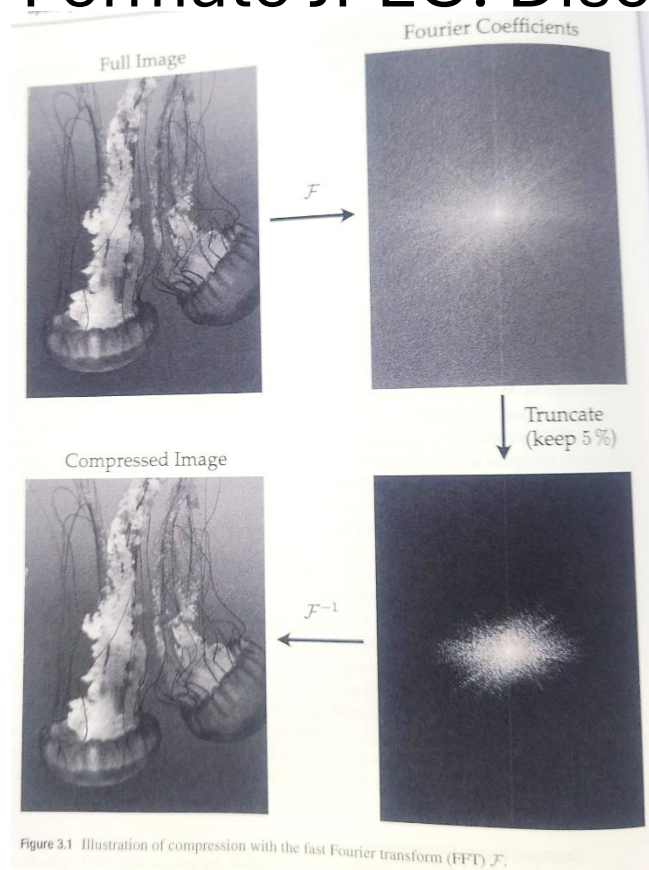
cv2.normalize(img_back, img_back, 0, 255,
cv2.NORM_MINMAX)
img_final = np.uint8(img_back)

comparison = np.hstack((img, img_final))
cv2.imshow("Original vs. 5.0% FFT Compression",
comparison)

cv2.waitKey(0)
```

## 4. Compresión de imágenes

- Formato JPEG: Discrete Cosine Transform (DCT)



<https://www.youtube.com/watch?v=Q2aEzeMDHMA>



## 5. Detección de líneas y círculos

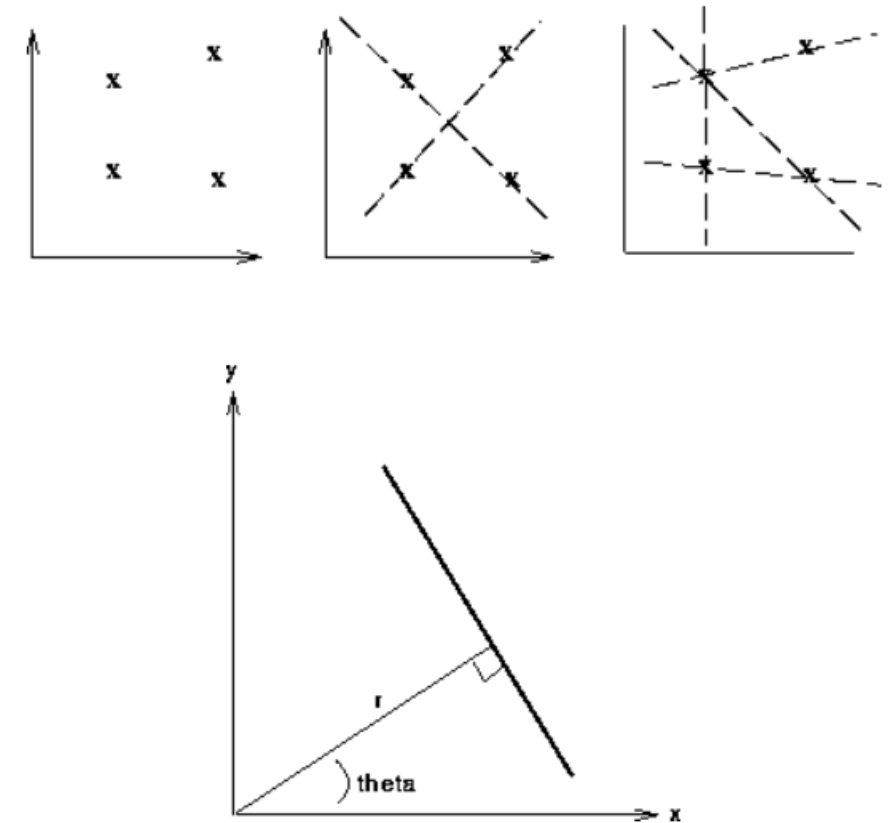
- Al momento hemos visto algoritmos para encontrar esquinas, bordes, manchas y contornos. Sin embargo, los resultados pueden contener ruido, lo cual dificulta identificar elementos primitivos como líneas y círculos.
- Para esto, se pueden utilizar las **Transformadas de Hough** para líneas y círculos.
- Se utilizan ecuaciones paramétricas para representar líneas y círculos.

$$x \cos \theta + y \sin \theta = r$$

$$(x - a)^2 + (y - b)^2 = r^2$$

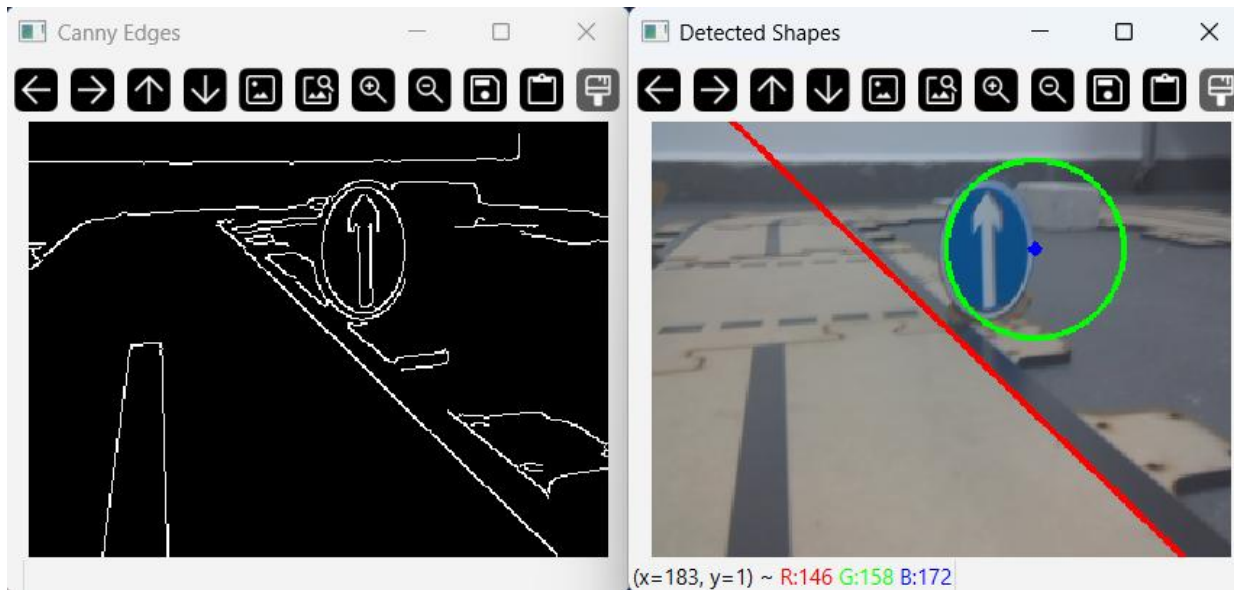
# 5. Detección de líneas y círculos

- Utilizando típicamente como punto de partida una imagen binaria, podremos encontrar líneas o círculos.
- Utiliza un algoritmo en donde se calculan las posibles líneas o círculos en la imagen, basados en los puntos x,y existentes.
- Para cada línea o círculo posible, se busca qué otros puntos x,y caen sobre las curvas, de manera que esos puntos darán votos a cada candidato de modelo de línea o círculo.
- Sólo prevalecen los modelos que tengan más votos.



# 5. Detección de líneas y círculos

- Hough lines and circles:



```
import cv2
import numpy as np

img = cv2.imread('image.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

blurred = cv2.GaussianBlur(gray, (5, 5), 0)

edges = cv2.Canny(blurred, 50, 150, apertureSize=3)

lines = cv2.HoughLines(edges, 1, np.pi/180, 200)

if lines is not None:
    for line in lines:
        rho, theta = line[0]
        a = np.cos(theta)
        b = np.sin(theta)
        x0 = a * rho
        y0 = b * rho
        x1 = int(x0 + 1000 * (-b))
        y1 = int(y0 + 1000 * (a))
        x2 = int(x0 - 1000 * (-b))
        y2 = int(y0 - 1000 * (a))
        cv2.line(img, (x1, y1), (x2, y2), (0, 0, 255), 2)

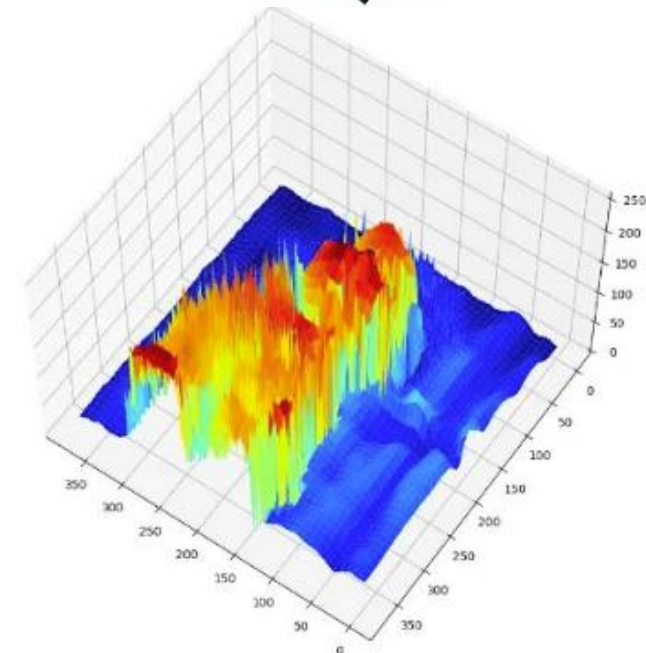
circles = cv2.HoughCircles(blurred, cv2.HOUGH_GRADIENT, dp=1.2,
minDist=100, param1=100, param2=30, minRadius=10, maxRadius=70)

if circles is not None:
    circles = np.uint16(np.around(circles))
    for i in circles[0, :]:
        cv2.circle(img, (i[0], i[1]), i[2], (0, 255, 0), 2)
        cv2.circle(img, (i[0], i[1]), 2, (255, 0, 0), 3)

cv2.imshow('Canny Edges', edges)
cv2.imshow('Detected Shapes', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

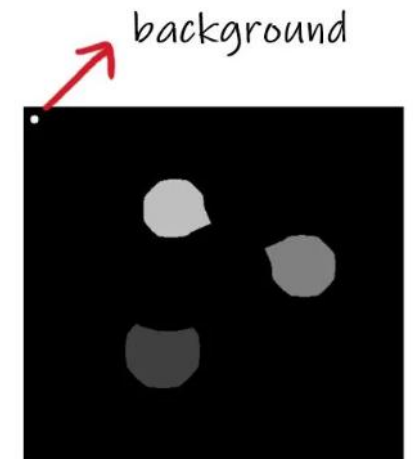
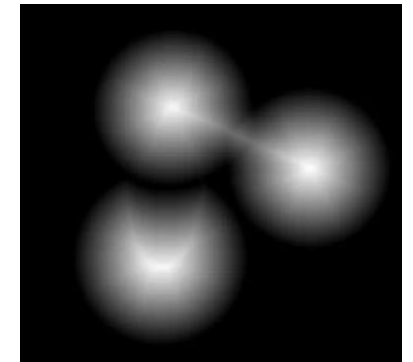
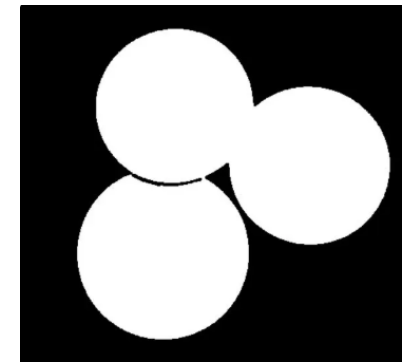
# 6. Algoritmo de Watershed

- En ocasiones se puede dificultar clasificar entre una instancia de un objeto y otra si los objetos se encuentran muy cercanos uno de otro. Es posible que 2 o más instancias se detecten como 1 sola.
- Si tomamos las intensidades de los pixeles, podemos clasificar 3 tipos de pixeles:
  - Pixeles en donde tenemos un mínimo.
  - Pixeles en donde si pusiéramos una gota de agua, la gota caerá al mínimo.
  - Pixeles en donde la gota puede caer con igual probabilidad a uno u otro mínimo.
- En el algoritmo de Watershed, queremos encontrar el tercer tipo de pixel, ya que es el que nos ayudará a dividir entre instancias.



## 6. Algoritmo de Watershed

- El algoritmo consiste en ir llenando de “agua” los mínimos hasta que los distintos “lagos” estén por unirse.
- Para seleccionar los mínimos que nos interesan, podemos preprocesar la imagen para tener un aproximado de nuestra instancia, o seleccionarla a mano. A esto se le llama “marcador”.
- Una idea simple, es usar la función `distanceTransform` de OpenCV, en donde nos devolverá una imagen en que representará la distancia del pixel blanco hasta el pixel negro más cercano. De ahí, tomamos los pixeles más “brillantes”.

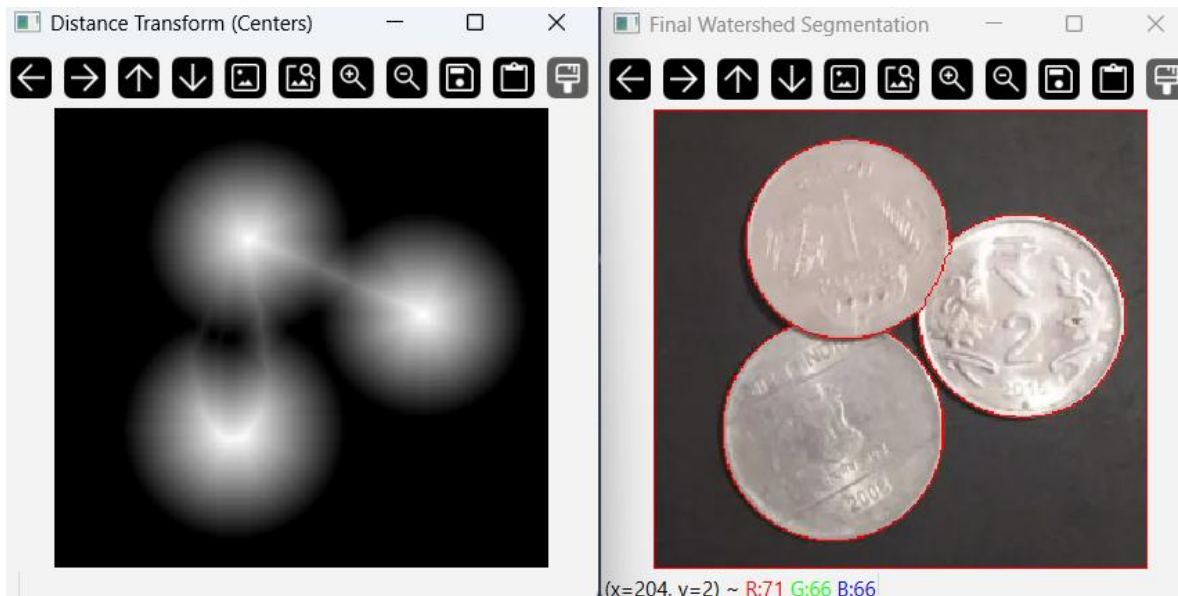


Markers



# 6. Algoritmo de Watershed

- Separando monedas:



```
import cv2
import numpy as np

img = cv2.imread('image.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

ret, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

kernel = np.ones((3,3), np.uint8)
closing = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel,
iterations=1)

dist_transform = cv2.distanceTransform(closing, cv2.DIST_L2, 3)

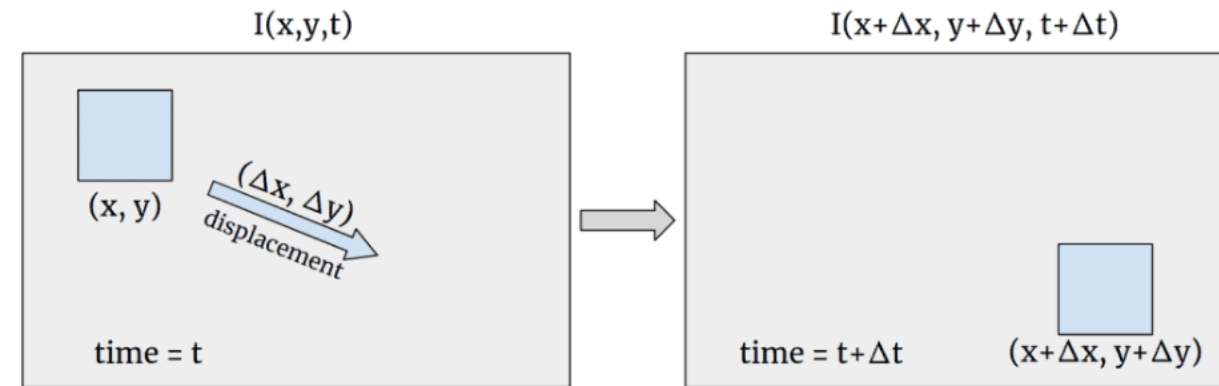
ret, dist1 = cv2.threshold(dist_transform, 0.6 *
dist_transform.max(), 255, 0)

markers = np.zeros(dist_transform.shape, dtype=np.int32)
dist_8u = dist1.astype('uint8')
contours, _ = cv2.findContours(dist_8u, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
for i in range(len(contours)):
    cv2.drawContours(markers, contours, i, (i+1), -1)
markers = cv2.circle(markers, (15,15), 5, len(contours)+1, -1)
markers = cv2.watershed(img, markers)
img[markers == -1] = [0,0,255]

# 9. Display
cv2.imshow('Distance Transform (Centers)',
dist_transform/dist_transform.max())
cv2.imshow('Final Watershed Segmentation', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

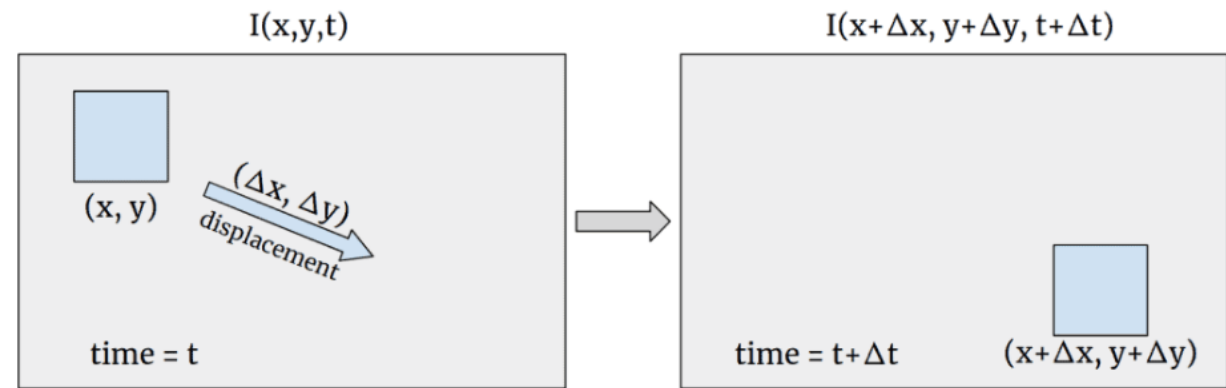
# 7. Flujo óptico

- Hasta el momento habíamos hecho procesamiento de imágenes de objetos estáticos. Sin embargo, en ocasiones deberemos de rastrear y calcular su movimiento entre un fotograma y otro.
- Para calcular el flujo óptico de algún objeto en movimiento aparente en un par de imágenes, se debe suponer lo siguiente:
  - La intensidad de los pixeles no deben cambiar entre la foto A y la foto B.
  - Existe persistencia temporal, es decir, los movimientos son suficientemente pequeños.
  - Hay coherencia espacial, ya que se espera que los pixeles pertenecientes al mismo objeto se muevan juntos



# 7. Flujo óptico

- Método de Lucas-Kanade:
  - Defina una **ecuación de flujo óptico**, donde se desea encontrar las **velocidades  $u$  y  $v$** .
  - $f$  representa los gradientes, que pueden ser calculados con **operador Sobel o Matriz Hessiana**.
  - Dado que se tienen 2 incógnitas y una sola ecuación, recurrimos a usar una ventana de 3x3 o 5x5 píxeles para utilizar **mínimos cuadrados** y resolver el problema.

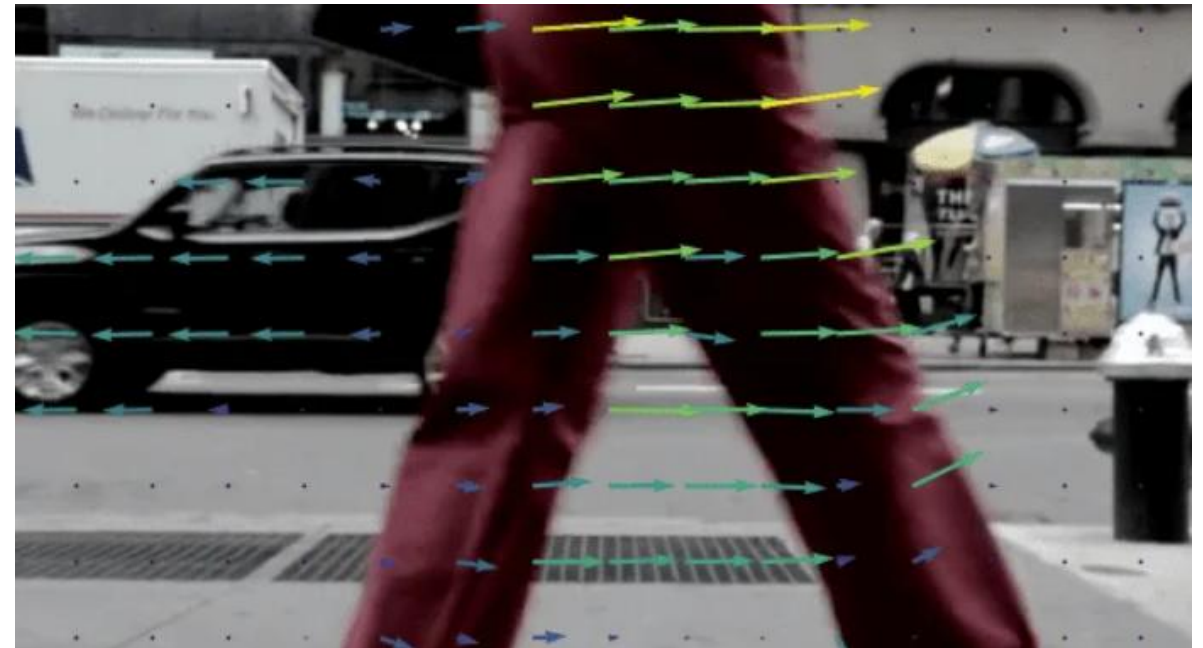


$$f_x u + f_y v + f_t = 0$$

- $f_x, f_y$ : Image gradients
- $f_t$ : The change in intensity over time.
- $u, v$ : The velocity we want to find.

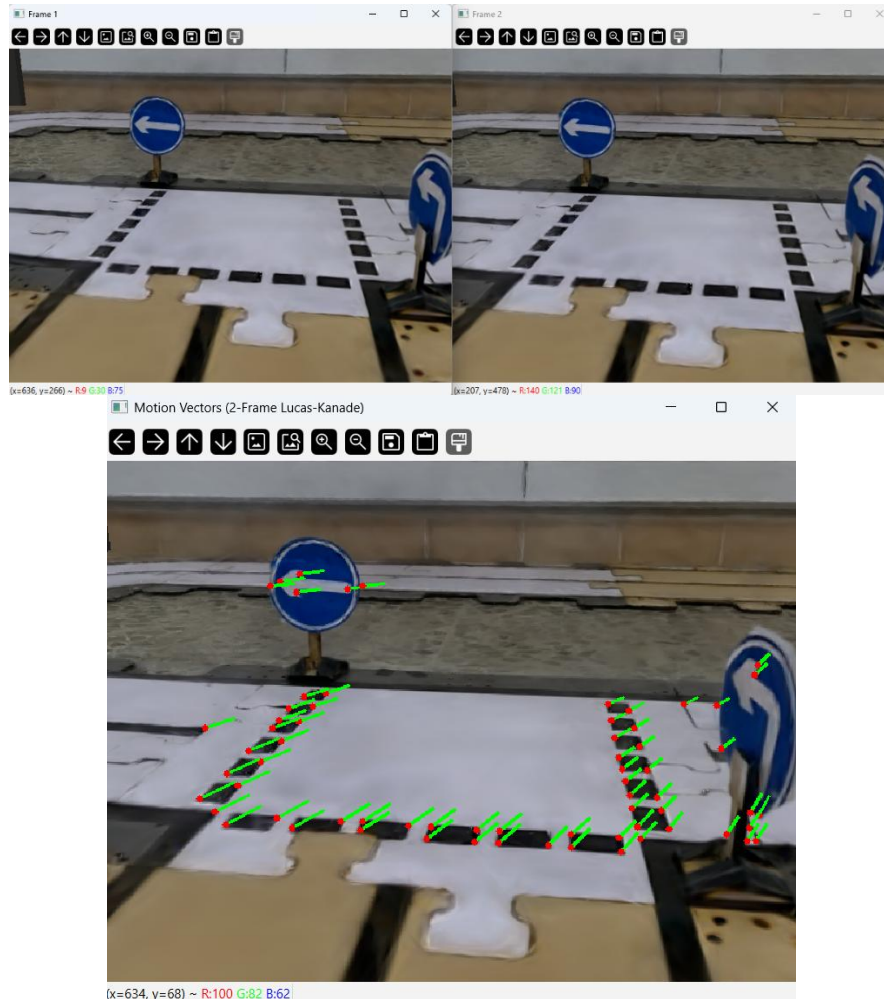
# 7. Flujo óptico

- Método de Lucas-Kanade:
  - Finalmente, podremos encontrar el flujo de movimiento para parches de píxeles, de los cuales podremos visualizar el movimiento expresado en vectores de velocidad.



# 7. Flujo óptico

- Estimación del movimiento:



```
import cv2
import numpy as np

img1 = cv2.imread('capture_0068.png')
img2 = cv2.imread('capture_0069.png')

old_gray = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
new_gray = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

feature_params = dict(maxCorners=100, qualityLevel=0.3,
minDistance=7, blockSize=7)
p0 = cv2.goodFeaturesToTrack(old_gray, mask=None,
**feature_params)

lk_params = dict(winSize=(21, 21), maxLevel=3,
criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10,
0.03))

p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray, new_gray, p0,
None, **lk_params)

if p1 is not None:
    good_new = p1[st == 1]
    good_old = p0[st == 1]

    output = img2.copy()
    for i, (new, old) in enumerate(zip(good_new, good_old)):
        a, b = new.ravel()
        c, d = old.ravel()

        cv2.line(output, (int(c), int(d)), (int(a), int(b)), (0,
255, 0), 2)
        cv2.circle(output, (int(a), int(b)), 3, (0, 0, 255), -1)

cv2.imshow('Frame 1', img1)
cv2.imshow('Frame 2', img2)
cv2.imshow('Motion Vectors (2-Frame Lucas-Kanade)', output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



# Fin

- Referencias principales:
  - Corke, P. (2023) Robotics, Vision and Control: Fundamental Algorithms in Python
  - Brunton, S. L. & Kutz, J. N. (2019) Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control
  - Nota: Algunos elementos de la presentación fueron preparados con el apoyo de TecGPT y Google Gemini 3.0 (2026).